HERS Events — Database Schema (public)

Generated from `app/db/schema.ts` + `relations.ts`. Arrows point from a foreign-key column to the primary key it references. The `neon_auth` schema is on page 2; query notes on page 3.

AUTH (BETTER AUTH)

account — 13 cols

PK	id			text
	account_id *			text
	provider_id *			text
FK	user_id *	→ user	cascade	text
	access_token			text
	refresh_token			text
	id_token			text
	access_token_expires_at			ts
	refresh_token_expires_at			ts
	scope			text
	password			text
	created_at *			ts
	updated_at *			ts

session — 9 cols

PK	id			text
	token *		unique	text
FK	user_id *	→ user	cascade	text
	expires_at *			ts
	ip_address			text
	user_agent			text
	impersonated_by			text
	created_at *			ts
	updated_at *			ts

verification — 6 cols

PK	id			text
	identifier *			text
	value *			text
	expires_at *			ts
	created_at *			ts
	updated_at *			ts

STAFF & ADMIN

staff_invites — 7 cols

PK	id			uuid
	email *			text
	token *		unique	text
FK	invited_by *	→ user		text
	used_at			tstz
	cancelled_at			tstz
	created_at *			tstz

settings — 4 cols

PK	key			text
	value *			text
FK	updated_by	→ user		text
	updated_at *			tstz

AUTH (BETTER AUTH)

user — 12 cols

PK	id			text
	name *			text
	email *		unique	text
	email_verified *			bool
	image			text
	role			text
	banned			bool
	ban_reason			text
	ban_expires			ts
	password_cleared *			bool
	created_at *			ts
	updated_at *			ts

STAFF & ADMIN

activity_log — 6 cols

PK	id			uuid
FK	actor_id	→ user		text
	action *			text
	details *			text
FK	event_id	→ events	set null	uuid
	created_at *			tstz

ANNOUNCEMENTS

announcements — 8 cols

PK	id			uuid
FK	event_id *	→ events	cascade	uuid
	title *			text
	body *			text
	pinned *			bool
FK	sent_by *	→ user		text
	created_at *			tstz
	updated_at *			tstz

announcement_reads — 3 cols

PK-FK	announcemen…	→ announcements	cascade	uuid
PK-FK	user_id	→ user	cascade	text
	read_at *			tstz

EVENTS

events — 12 cols

PK	id			uuid
	name *			text
	description *			text
	start_date *			date
	end_date *			date
	time_zone *			text
	location			text
	virtual_details			text
	access_expiry *			date
	access_key *		unique	text
FK	created_by *	→ user		text
	created_at *			tstz

event_links — 7 cols

PK	id			uuid
FK	event_id *	→ events	cascade	uuid
	label *			text
	url *			text
	sort_order *			int
	visible_from			tstz
	visible_until			tstz

AGENDA

tracks — 4 cols

PK	id			uuid
FK	event_id *	→ events	cascade	uuid
	name *			text
	sort_order *			int

speakers — 9 cols

PK	id			uuid
FK	event_id *	→ events	cascade	uuid
	name *			text
	title			text
	organization			text
	bio *			text
	email			text
	linkedin			text
	website			text

sessions_agenda — 9 cols

PK	id			uuid
FK	event_id *	→ events	cascade	uuid
FK	track_id	→ tracks	set null	uuid
	title *			text
	starts_at *			tstz
	ends_at *			tstz
	location			text
	virtual_link			text
	description *			text

session_speakers — 3 cols

PK-FK	session_id	→ sessions_agenda	cascade	uuid
PK-FK	speaker_id	→ speakers	cascade	uuid
	sort_order *			int

ATTENDEES & CHECK-IN

check_in_points — 4 cols

PK	id			uuid
FK	event_id *	→ events	cascade	uuid
	label *			text
	date			date

check_ins — 5 cols

PK	id			uuid
FK	point…	→ check_in_points	cascade unique	uuid
FK	enrollm…	→ enrollments	cascade unique	uuid
	checked_in_at *			tstz
FK	checked_in_by *	→ user		text

enrollments — 11 cols

PK	id			uuid
FK	event_id *	→ events	cascade unique	uuid
FK	user_id	→ user		text
	invited_email *		unique	text
	status *			enum
	individual_expiry			date
	display_name *			text
	title			text
	organization			text
	linkedin			text
	created_at *			tstz

subcommittees — 4 cols

PK	id			uuid
FK	event_id *	→ events	cascade	uuid
	name *			text
	group_me_url *			text

subcommittee_members — 2 cols

PK-FK	subcommitte…	→ subcommittees	cascade	uuid
PK-FK	enrollment_id	→ enrollments	cascade	uuid

How to read this

PK primary key **FK** foreign key * NOT NULL

● FK column (many side) → arrow at the referenced PK (one side)

—— required FK - - - - nullable FK

wire to **user** wire to **events** other

cascade child rows deleted with parent

set null FK cleared when parent deleted

no tag on an FK = delete of parent is **blocked**

unique unique alone or as part of a composite

Types: `ts` timestamp, `tstz` timestamp with tz.

`status` enum = invited | active | revoked

neon_auth schema (managed by Neon Auth)

A separate Postgres schema provisioned by Neon Auth. It is **not linked** to the public tables — the app's FKs all point at `public.user` (text ids), not `neon_auth.user` (uuid ids).

account — 13 cols

PK	id			uuid
	accountId *			text
	providerId *			text
FK	userId *	→ user	cascade	uuid
	accessToken			text
	refreshToken			text
	idToken			text
	accessTokenExpiresAt			tstz
	refreshTokenExpiresAt			tstz
	scope			text
	password			text
	createdAt *			tstz
	updatedAt *			tstz

session — 10 cols

PK	id			uuid
	token *		unique	text
FK	userId *	→ user	cascade	uuid
	expiresAt *			tstz
	ipAddress			text
	userAgent			text
	impersonatedBy			text
	activeOrganizationId			text
	createdAt *			tstz
	updatedAt *			tstz

user — 11 cols

PK	id			uuid
	name *			text
	email *		unique	text
	emailVerified *			bool
	image			text
	role			text
	banned			bool
	banReason			text
	banExpires			tstz
	createdAt *			tstz
	updatedAt *			tstz

member — 5 cols

PK	id			uuid
FK	organizationI…	→ organization	cascade	uuid
FK	userId *	→ user	cascade	uuid
	role *			text
	createdAt *			tstz

invitation — 8 cols

PK	id			uuid
FK	organizationI…	→ organization	cascade	uuid
FK	inviterId *	→ user	cascade	uuid
	email *			text
	role			text
	status *			text
	expiresAt *			tstz
	createdAt *			tstz

organization — 6 cols

PK	id			uuid
	name *			text
	slug *		unique	text
	logo			text
	metadata			text
	createdAt *			tstz

verification — 6 cols

PK	id			uuid
	identifier *			text
	value *			text
	expiresAt *			tstz
	createdAt *			tstz
	updatedAt *			tstz

jwks — 7 cols

PK	id			uuid
	publicKey *			text
	privateKey *			text
	alg			text
	crv			text
	createdAt *			tstz
	expiresAt			tstz

project_config — 12 cols

PK	id			uuid
	name *			text
	endpoint_id *		unique	text
	trusted_origins *			jsonb
	social_providers *			jsonb
	email_provider			jsonb
	email_and_password			jsonb
	allow_localhost *			bool
	plugin_configs			jsonb
	webhook_config			jsonb
	created_at *			tstz
	updated_at *			tstz

Notes for writing queries & server functions

Things in the schema that will shape your code — cascade rules, uniqueness, join tables and the Drizzle relation names you can use in `db.query.*.findMany({ with: … })`.

The two hubs

- **events** owns almost everything: enrollments, check_in_points, subcommittees, announcements, tracks, sessions_agenda, speakers, event_links. Deleting an event **cascades** to all of them (and onward to check_ins, subcommittee_members, announcement_reads, session_speakers).
- **user** is the actor: who created an event, sent an announcement, checked someone in, invited staff, changed a setting, or wrote an activity_log row.

Deleting a user will fail if…

These FKs to `user.id` have **no ON DELETE** rule (Postgres default = NO ACTION), so the delete is rejected while any row still points at the user:

- `events.created_by`
- `announcements.sent_by`
- `check_ins.checked_in_by`
- `staff_invites.invited_by`
- `enrollments.user_id` (nullable)
- `settings.updated_by` (nullable)
- `activity_log.actor_id` (nullable)

Only `account`, `session` and `announcement_reads` cascade. Plan to reassign / null these out, or ban instead of delete.

Enrollment lifecycle

- An enrollment is created from an **email** first: `invited_email` is required, `user_id` is nullable and filled in when the attendee signs up.
- `status` enum: `invited` → `active` → `revoked` (default `invited`).
- Unique on `(event_id, invited_email)` — use `onConflictDoNothing` / `DoUpdate` for bulk invites.
- Access: `events.access_expiry` is the event-wide cutoff; `enrollments.individual_expiry` can override per person.
- `events.access_key` is unique — a natural lookup key for join links.

Check-ins

- Each event has one or more **check_in_points** (default label "Arrival", optional `date` for multi-day events).
- A check-in links a point + an **enrollment** (not a user) + the staff user who scanned it.
- Unique on `(point_id, enrollment_id)` → one check-in per person per point. Duplicate scans will throw unless you handle the conflict.

Join tables (composite PKs, no id column)

- `announcement_reads` (announcement_id, user_id) + `read_at`
- `session_speakers` (session_id, speaker_id) + `sort_order`
- `subcommittee_members` (subcommittee_id, enrollment_id)

Insert with `onConflictDoNothing()` to make "mark read" / "add member" idempotent.

Agenda

- `sessions_agenda` = a talk/slot (the table name avoids clashing with the auth `session` table). Indexed on `(event_id, starts_at)` — order by `starts_at`.
- `track_id` is optional and **set null** when a track is deleted.
- Speakers belong to an event and attach to sessions through `session_speakers`.

Drizzle relation names (relations.ts)

```
events:   enrollments, checkInPoints, subcommittees,
          announcements, tracksEventId, sessionsAgenda,
          speakers, eventLinks, activityLogs,
          user (= creator)
enrollments: event, user, checkIns,
          subcommitteeMembers, subcommittees (via join)
sessionsAgenda: event, track, sessionSpeakers,
          speakers (via join)
announcements: event, user (= sender),
          announcementReads, users (readers, via join)
checkIns: checkInPoint, enrollment, user (= staff)
user:     enrollments, eventsCreatedBy,
          announcementsSentBy, checkIns, staffInvites, …
```

Note: an event's tracks are `tracksEventId`, not `tracks` (`tracksViaSessionsAgenda` goes through the agenda instead).

```
db.query.events.findFirst({
  where: { id },
  with: {
    sessionsAgenda: { with: { speakers: true, track: true } },
    enrollments: { with: { checkIns: true } },
  },
})
```

Auth / admin tables

- `user`, `account`, `session`, `verification` are the Better Auth tables (text ids). `user.role`, `banned`, `ban_*` come from the admin plugin; `password_cleared` is app-specific.
- `staff_invites`: unique `token`; an invite is open while `used_at` and `cancelled_at` are both null.
- `settings`: key/value store, `key` is the PK — upsert on it.
- `activity_log.event_id` is **set null** when the event is deleted, so the log survives.